

# API DOCUMENTATION

## Informasi Umum

**Base URL (Production):** <https://project-svara-production.up.railway.app>

**Format Data:** JSON (request maupun response)

**Autentikasi:** Belum menggunakan token (JWT) — status login disimpan pada sessionStorage sisi klien setelah proses login berhasil. Endpoint API sendiri tidak memvalidasi token pada header request.

**CORS:** Diaktifkan untuk seluruh origin (@CrossOrigin("\*") pada Svara2Controller), sehingga API dapat diakses dari domain manapun.

## Daftar Endpoint

| Method | Endpoint                         | Fungsi                              |
|--------|----------------------------------|-------------------------------------|
| POST   | /admin/login                     | Otentikasi login Admin              |
| POST   | /register                        | Registrasi akun User baru           |
| POST   | /user/login                      | Otentikasi login User               |
| GET    | /users                           | Mengambil seluruh data User         |
| GET    | /users/{id}                      | Mengambil detail satu User          |
| PUT    | /users/{id}                      | Memperbarui data User               |
| DELETE | /users/{id}                      | Menghapus data User                 |
| POST   | /pembayaran                      | Menyimpan transaksi pembelian tiket |
| GET    | /pembayaran/riwayat?namaUser=... | Riwayat pembelian tiket per User    |

## Detail Setiap Endpoint

**POST**    **/admin/login**

### Request Body

| Field    | Tipe   | Wajib |
|----------|--------|-------|
| email    | String | Ya    |
| Field    | Tipe   | Wajib |
| password | String | Ya    |

```
{
  "email": "admin@svara.com",
  "password": "admin123"
}
```

**Response 200 OK** — kredensial valid, mengembalikan data Admin (tanpa password disaring secara eksplisit):

```
{
  "id": 1,
  "email": "admin@svara.com",
  "password": "admin123"
}
```

**Response 401 Unauthorized** — kredensial tidak valid:

```
"Email atau password salah"
```

**POST**    **/register**

---

### Request Body

| Field        | Tipe   | Wajib |
|--------------|--------|-------|
| nama         | String | Ya    |
| email        | String | Ya    |
| password     | String | Ya    |
| nomorTelepon | String | Ya    |
| tanggalLahir | String | Ya    |

```
{
  "nama": "Keisya Putri",
  "email": "keisya@mail.com",
  "password": "rahasia123",
  "nomorTelepon": "08123456789",
  "tanggalLahir": "2003-05-10"
}
```

**Response 200 OK** — mengembalikan objek User yang tersimpan lengkap dengan id hasil auto-increment:

```
{
  "id": 12,
  "nama": "Keisya Putri",
  "email": "keisya@mail.com",
  "password": "rahasia123",
  "nomorTelepon": "08123456789",
}
```

```
{
  "tanggalLahir": "2003-05-10"
}
```

**POST**    **/user/login**

---

### Request Body

| Field    | Tipe   | Wajib |
|----------|--------|-------|
| email    | String | Ya    |
| password | String | Ya    |

```
{
  "email": "keisya@mail.com",
  "password": "rahasia123"
}
```

**Response 200 OK** — mengembalikan data User yang login. **Response 401 Unauthorized** — dengan body pesan "Email atau password salah" jika kredensial tidak sesuai (perilaku identik dengan /admin/login).

**GET**    **/users**

---

### Response 200 OK

```
[
  {
    "id": 1,
    "nama": "Keisya Putri",
    "email": "keisya@mail.com",
    "password": "rahasia123",
    "nomorTelepon": "08123456789",
    "tanggalLahir": "2003-05-10"
  },
  { "id": 2, "...": "..." }
]
```

**GET**    **/users/{id}**

---

**Path Parameter** — id: int, ID User yang dicari.

**Response 200 OK** — mengembalikan objek User. **Response 404 Not Found** — dengan body pesan "User tidak ditemukan" jika ID tidak ada.

**PUT**    **/users/{id}**

---

**Path Parameter** — id: int.

**Request Body** — sama seperti pada /register (nama, email, password, nomorTelepon, tanggalLahir); field yang tidak ingin diubah tetap harus dikirim ulang karena Service melakukan penggantian objek secara langsung.

**Response 200 OK** — mengembalikan objek User terbaru. **Response 404 Not Found** — jika ID tidak ditemukan.

**DELETE** /users/{id}

---

**Response 200 OK** — body berupa teks "User berhasil dihapus". **Response 404 Not Found** — body berupa teks "User tidak ditemukan" jika ID tidak ada.

**POST** /pembayaran

---

### Request Body

| Field            | Tip    | Wajib | Keterangan                                                   |
|------------------|--------|-------|--------------------------------------------------------------|
| namaKonser       | String | Ya    | Nama konser yang dibeli tiketnya                             |
| kategoriTiket    | String | Ya    | Kategori/kelas tiket                                         |
| namaUser         | String | Ya    | Nama pembeli (dicocokkan manual dengan field nama pada User) |
| jumlahTiket      | int    | Ya    | Jumlah tiket yang dibeli                                     |
| metodePembayaran | String | Ya    | Metode pembayaran yang dipilih                               |
| totalBayar       | double | Ya    | Total nominal pembayaran                                     |

```
{
  "namaKonser": "Konser Musik Indonesia",
  "kategoriTiket": "VIP",
  "namaUser": "Keisya Putri",
  "jumlahTiket": 2,
  "metodePembayaran": "Transfer Bank",
  "totalBayar": 1500000
}
```

**Response 200 OK** — mengembalikan objek Pembayaran yang tersimpan, lengkap dengan id auto-increment dan tanggalBeli yang diisi otomatis oleh server (format "d MMMM yyyy, HH:mm", locale id-ID):

```
{
  "id": 5,
  "namaKonser": "Konser Musik Indonesia",
  "kategoriTiket": "VIP",
  "namaUser": "Keisya Putri",
  "tanggalBeli": "23 Juli 2026, 10:45",
}
```

```
"jumlahTiket": 2,  
"metodePembayaran": "Transfer Bank",  
"totalBayar": 1500000  
}
```

**GET** `/pembayaran/riwayat?namaUser=...`

---

**Query Parameter** — `namaUser`: String, wajib. Contoh: `GET /pembayaran/riwayat?namaUser=Keisya`

**Response 200 OK** — mengembalikan array Pembayaran milik `namaUser` tersebut, terurut terbaru lebih dahulu:

```
[  
  {  
    "id": 5,  
    "namaKonser": "Konser Musik Indonesia",  
    "kategoriTiket": "VIP",  
    "namaUser": "Keisya",  
    "tanggalBeli": "23 Juli 2026, 10:45",  
    "jumlahTiket": 2,  
    "metodePembayaran": "Transfer Bank",  
    "totalBayar": 1500000  
  }  
]
```

## Catatan Keamanan

- Password disimpan dan dikembalikan dalam bentuk plain text pada response (mis. `/admin/login`, `/register`) — belum menggunakan hashing (BCrypt) maupun penyaringan field sensitif sebelum dikirim ke klien.
- Tidak ada mekanisme token (JWT) pada endpoint mana pun — setelah login berhasil, status sesi hanya disimpan di `sessionStorage` browser, sehingga API sesungguhnya dapat diakses langsung tanpa otentikasi ulang oleh siapa pun yang mengetahui endpoint-nya.
- Tidak ada validasi input (Jakarta Validation) pada request body — field kosong atau format tidak sesuai (misalnya email tanpa format valid) tetap akan diproses dan disimpan ke database.
- Rekomendasi pengembangan: tambahkan Spring Security + JWT, DTO dengan validasi input, dan penyaringan field password dari response — sejalan dengan rencana pengembangan pada Bagian H dokumen UAS.